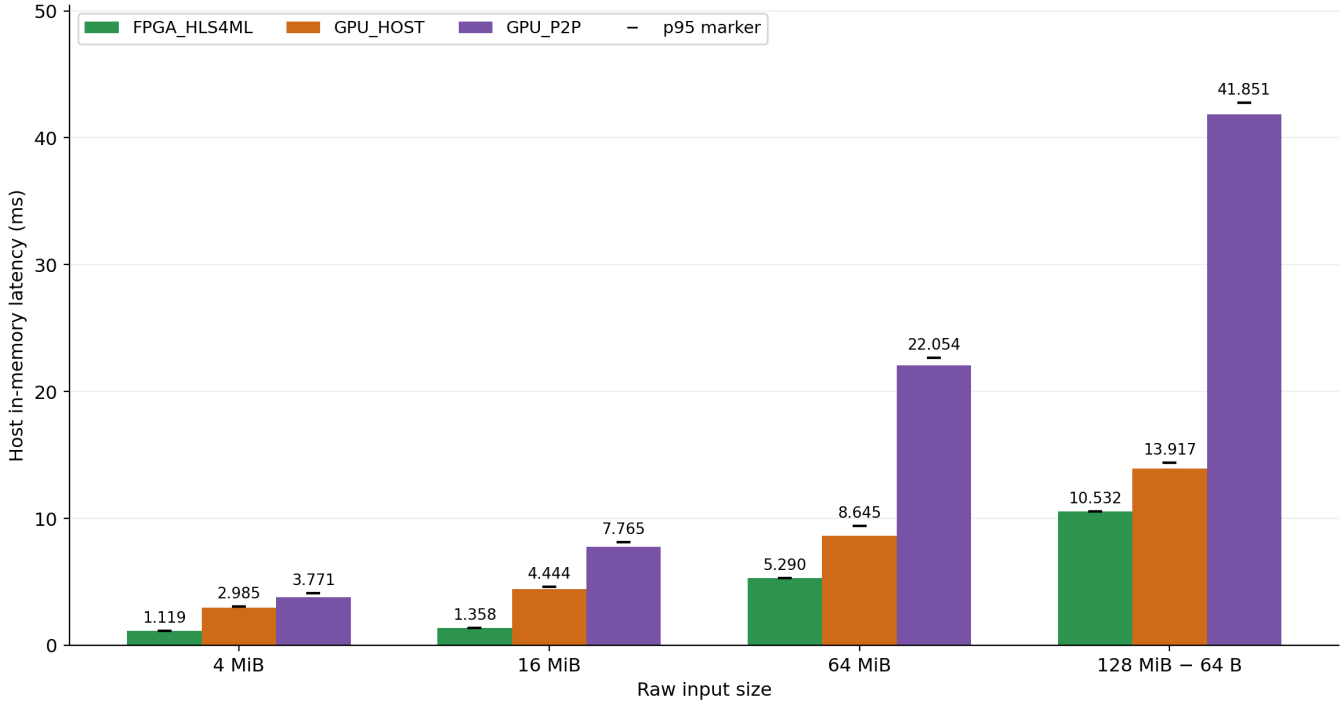
hls4ml-checker: host in-memory latency

Figure 1 · Synthetic inputs · Accepted campaign: 19 October 2026

Host in-memory means the bitstream is already prepared in host RAM when timing starts. Every trial still transfers the complete input and ends when the host observes the FPGA verdict. File reading and host-buffer preparation are outside this timing window.



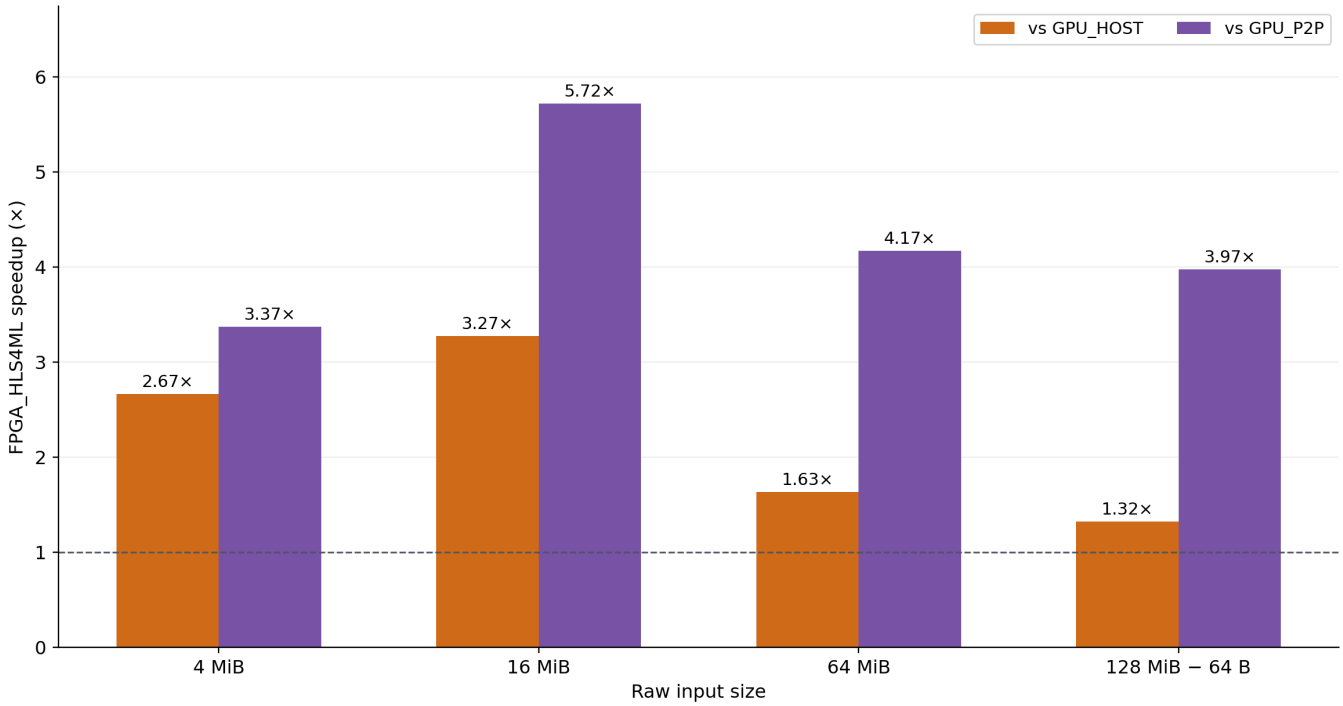
At 4 MiB: FPGA_HLS4ML takes 1.119 ms, GPU_HOST 2.985 ms, and GPU_P2P 3.771 ms. At the largest input the respective medians are 10.532, 13.917, and 41.851 ms.

Bars show measured medians; black ticks show p95. FPGA_HLS4ML is the Resource-optimized streaming checker. GPU_HOST sends host input to both GPU and FPGA; GPU_P2P forwards input through the FPGA and uses autonomous result polling.

Comparison scope: FPGA W8A8 versus GPU FP32, one U55C/MI210 pairing, 200 measured trials after 20 warmups per input and implementation. These are implemented-system timings, not equal-precision kernel benchmarks. The main figures use four synthetic sizes; real bitstreams appear only in the appendix.

hls4ml-checker speedup over GPU paths

Figure 2 · Synthetic inputs · Higher is better for FPGA_HLS4ML



Speedup = GPU median latency / FPGA_HLS4ML median latency. Each ratio compares the same input size and host in-memory timing boundary. A value of 1× indicates equal latency; values above 1× favor the checker.

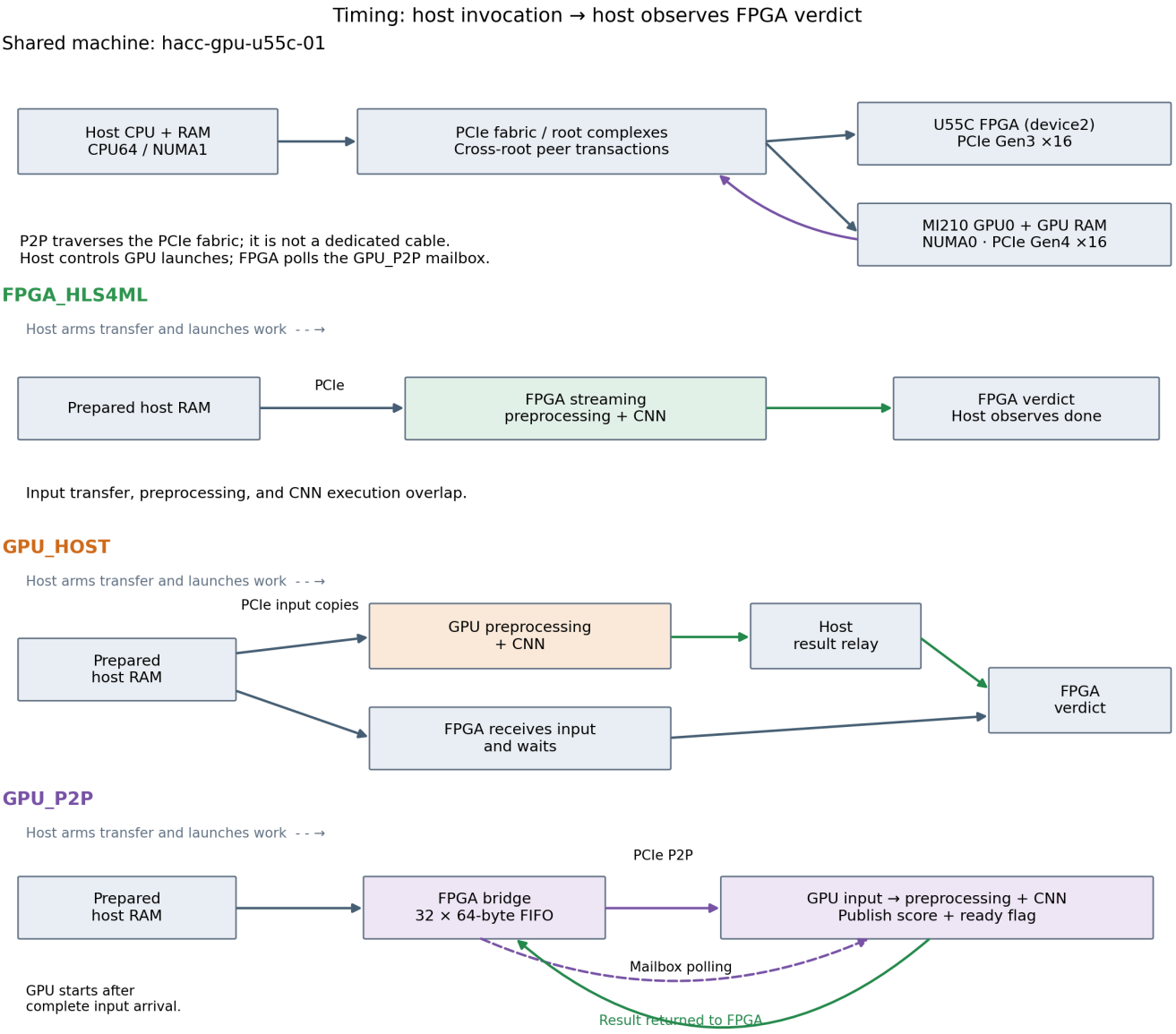
Against GPU_HOST, the checker delivers 2.67× speedup at 4 MiB and 1.32× at the largest input. Against GPU_P2P, the corresponding speedups are 3.37× and 3.97×.

The benefit depends on the complete transfer and execution path. GPU kernel execution alone is much shorter than GPU end-to-end latency; Figures 4–6 separate observable host phases, execution overlap, and diagnostic device timings.

Ratios are formed from independent-run medians. They are not paired per-trial ratios or confidence intervals. The original full report remains the complete workload record.

Benchmark setup and data paths

Figure 3 · Shared physical setup and logical paths

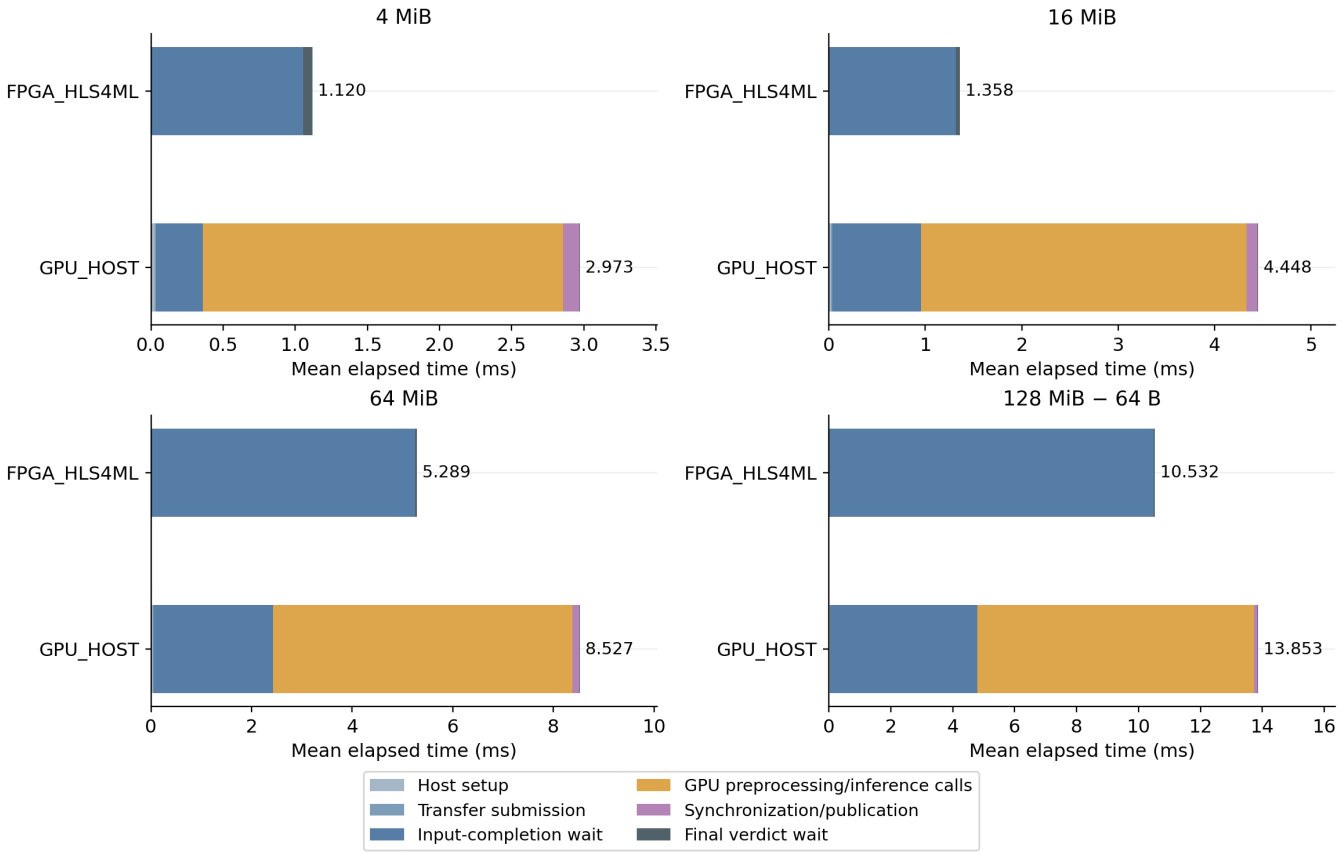


The host arms transfers and launches GPU work. Solid arrows carry input data; green arrows indicate verdict movement; dashed arrows denote control/polling. GPU_P2P reads a readiness field and result payload directly from GPU memory through the PCIe fabric.

The FPGA_HLS4ML path streams pixels into the CNN while input arrives. GPU paths start their preprocessing/inference calls after GPU input readiness. The P2P bridge forwards incrementally through a 2 KiB payload FIFO; it does not buffer the complete input on the FPGA.

Where invocation-to-verdict time is spent

Figure 4 · FPGA_HLS4ML versus GPU_HOST · Host in-memory



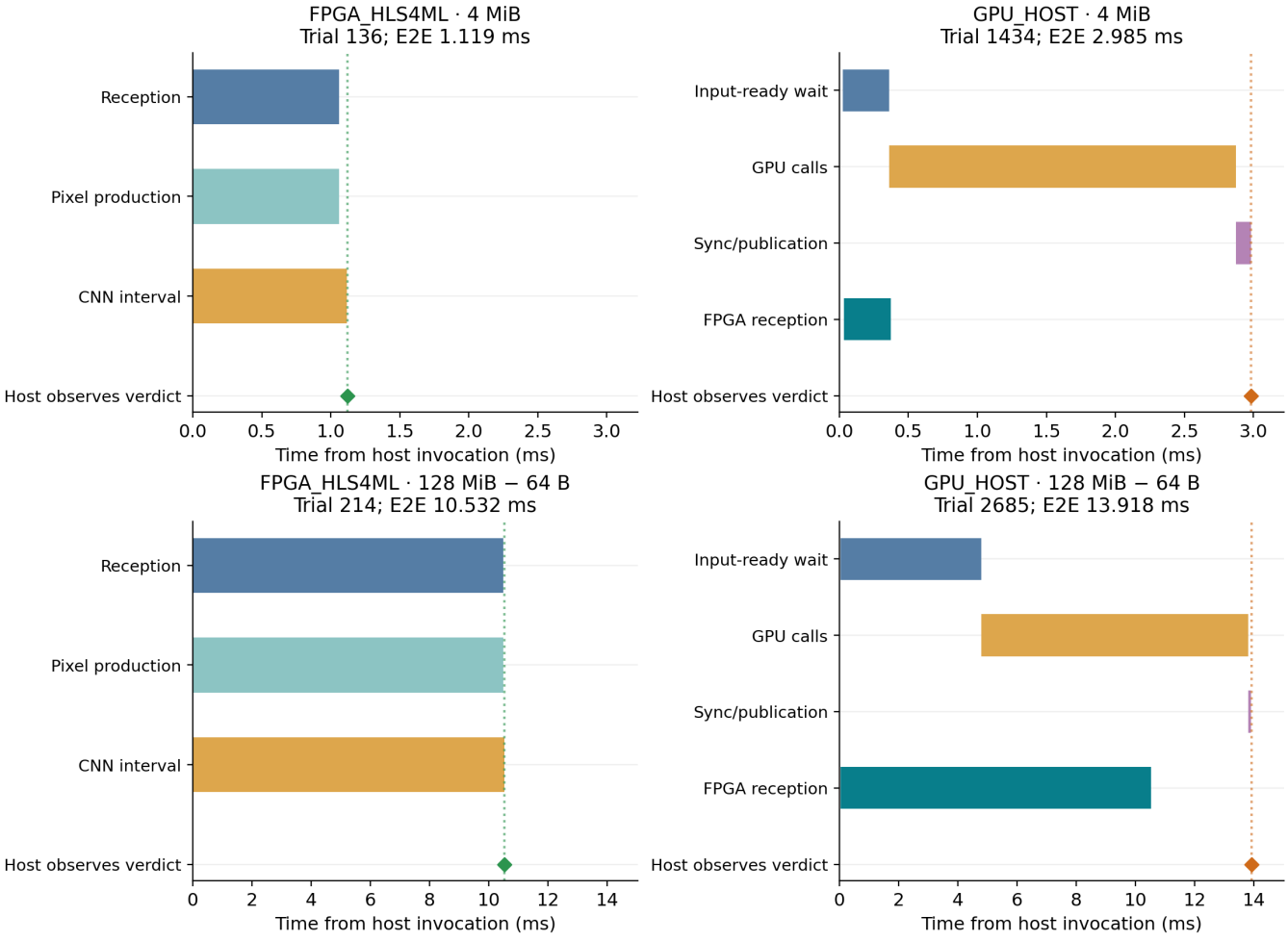
Each bar adds consecutive host-observed intervals from invocation to verdict. Segments are **means** so they sum exactly to mean total latency; headline figures use medians. Tiny setup and submission intervals are retained in the exported data.

For FPGA_HLS4ML, input-completion wait ends when the host observes the DMA acknowledgement. The subsequent verdict wait can include remaining FPGA work. Neither interval is a pure compute measurement: preprocessing and CNN execution overlap transfer.

For GPU_HOST, input-completion wait ends when its GPU copy is ready. GPU preprocessing/inference calls include useful execution, dispatch, and gaps. Synchronization/publication can include waiting for unfinished GPU work, obtaining the score, and relaying it to the FPGA. An absent GPU-only segment in the checker bar does not mean its CNN takes zero time.

Why overlap changes the critical path

Figure 5 · Representative measured host in-memory trials



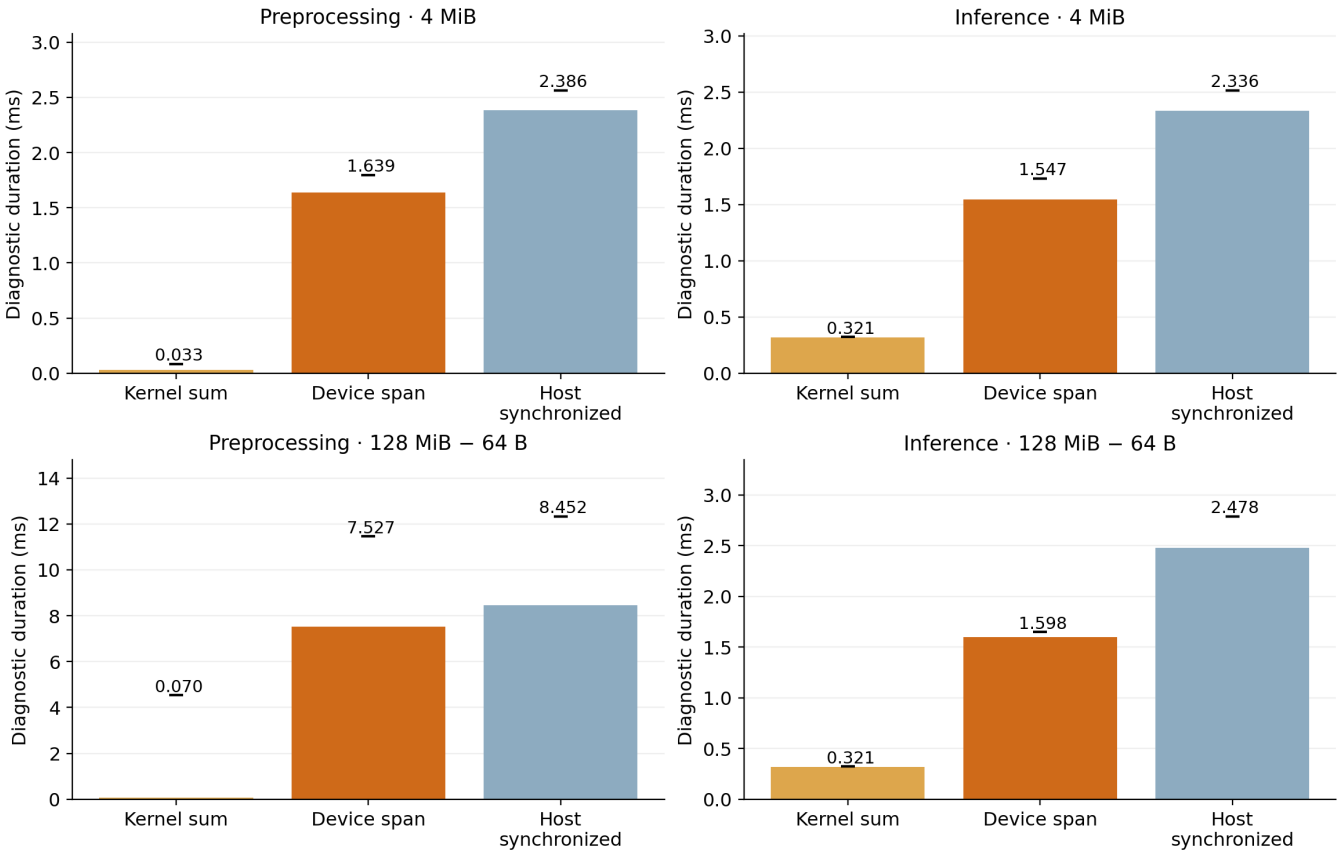
Panels within each size share an X-axis limit. Each uses an actual trial nearest that implementation's median total, identified in the title. Diamonds mark host-observed completion; black endpoint marks show FPGA clock-alignment uncertainty, generally smaller than the plotted line width.

FPGA_HLS4ML receives bytes, produces pixels, and consumes them in the CNN concurrently. GPU_HOST waits for GPU input readiness before launching preprocessing/inference. Its separate FPGA input transfer can overlap GPU activity. The checker's advantage is therefore an execution-path result, not a claim that each individual FPGA operation is faster.

Parallel lanes overlap and must not be added. The GPU HIP copy duration has no recorded host-aligned start, so this figure shows the host input-ready interval instead of inventing a device-copy timeline.

GPU kernel time is not system latency

Figure 6 · GPU_HOST-only diagnostic profiling



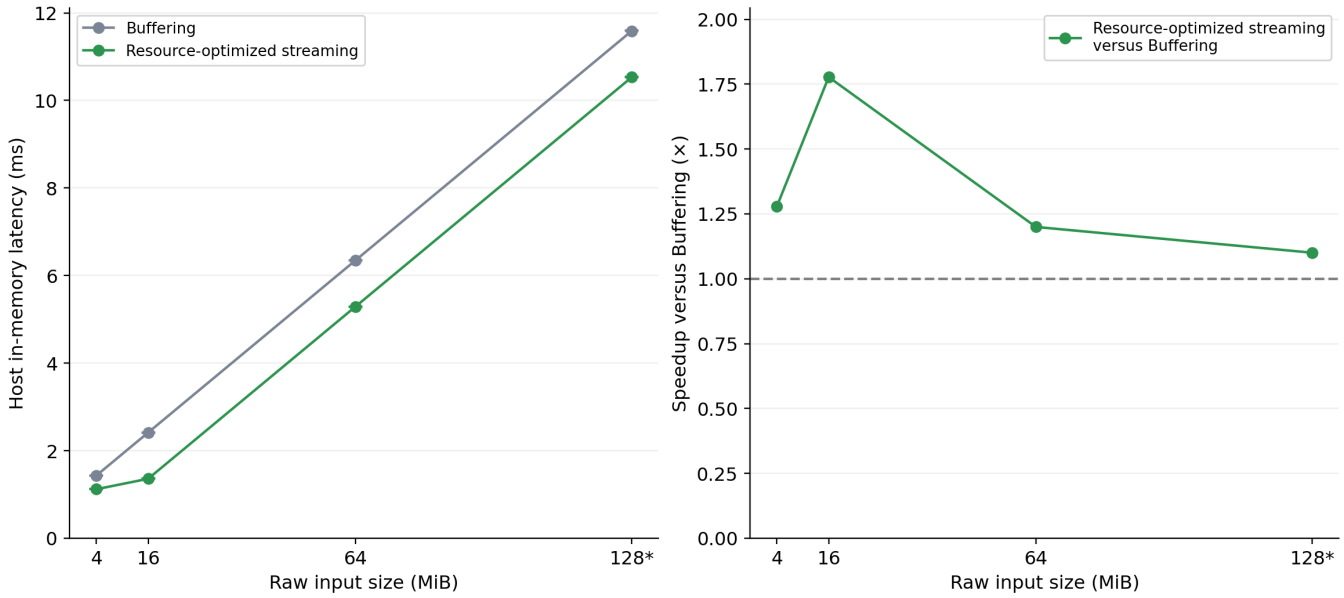
Kernel sum: adds attributed GPU kernel durations, excluding gaps. **Device span:** first kernel start to last kernel completion, including gaps. **Host synchronized:** host timing around the phase and explicit output synchronization/materialization. Bars show medians; black ticks show p95.

The larger spans show time beyond accumulated kernel execution. They can include scheduling gaps, dispatch, and synchronization, but the trace does not identify every underlying cause. Kernel sums may also overlap if kernels execute concurrently.

These are separate, explicitly synchronized profiling runs: 20 trials per workload, with profiling/first-use effects possible. They are not parts to add together or subtract from normal host in-memory latency. They explain why a short kernel duration does not guarantee a short application response.

Checker optimization: latency impact

Figure 7 · Buffering versus Resource-optimized streaming



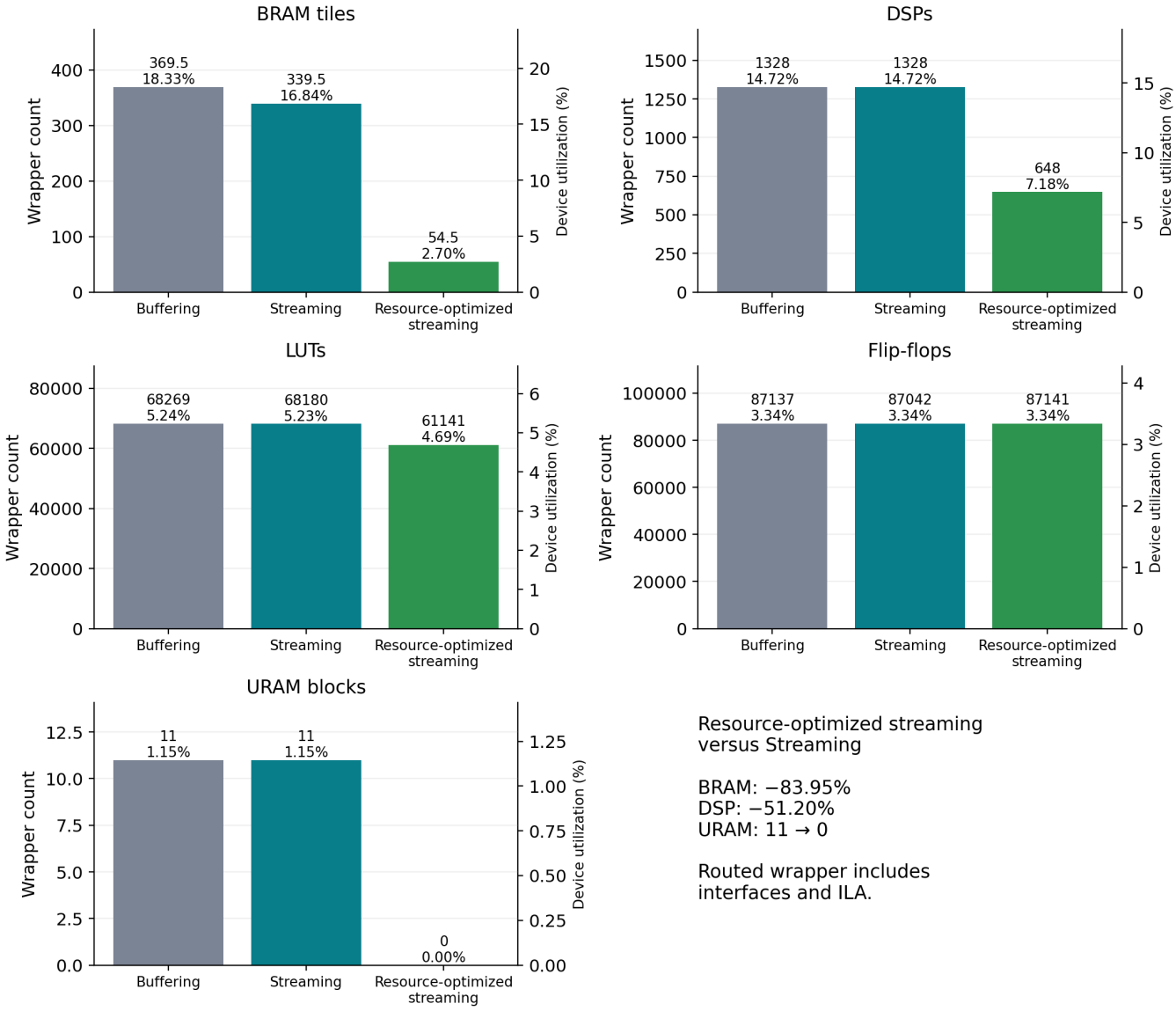
At 4 MiB, Buffering takes 1.431 ms and Resource-optimized streaming takes 1.119 ms: a 1.28× speedup. At the largest input, the speedup is 1.10×.

The left panel compares host in-memory medians, with p95 ticks. The right divides Buffering median by Resource-optimized streaming median. Connecting lines guide the eye between the four measured synthetic sizes; intermediate sizes were not measured. The final point is 128 MiB – 64 B.

Resource-optimized streaming combines native hls4ml FIFO optimization (profiling capacity 1024) with convolution reuse factors 72/72/288/288 for conv3/4/5/6. It retains the frozen quantized model, streaming preprocessing, and timestamp interface. The temporary profiling capacity is not a uniform final FIFO depth.

Checker optimization: resource impact

Figure 8 · FPGA variants only

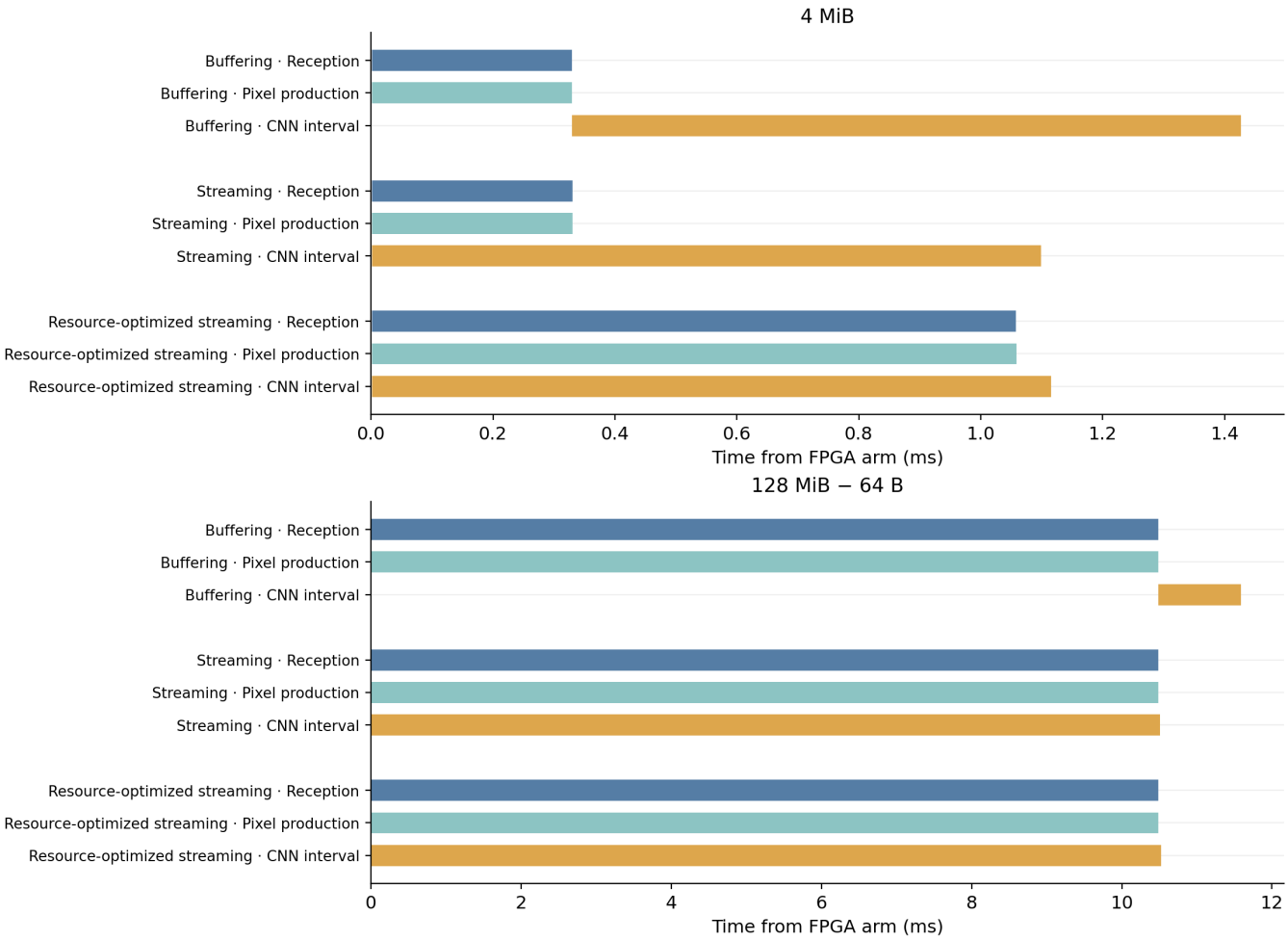


Relative to Streaming, Resource-optimized streaming reduces wrapper BRAM from **339.5 to 54.5 tiles** (83.95%) and DSPs from **1328 to 648** (51.20%). URAM falls from 11 blocks to zero.

Counts are routed user-wrapper resources, including interfaces and ILA. Right axes and labels use total U55C capacity as the denominator. These are not full-design counts: the shell occupies additional resources. Resource changes come from build reports; latency comes from measured board runs.

How streaming hides preprocessing and CNN time

Figure 9 · Same FPGA timestamp definitions across three variants



Buffering waits for the complete input and pixel frame before CNN consumption. Streaming starts CNN consumption as pixels become available. Resource-optimized streaming preserves this overlap while reducing memory and multiplier resources.

All lanes use the FPGA clock relative to arm. Pixel production begins at the first emitted pixel; reception starts with the first accepted payload byte. The CNN interval ends at the score and includes stalls waiting for incoming pixels. A long CNN interval at large sizes is not standalone arithmetic time. Selected trial IDs are in data/timelines.json.

Measurement definitions and evidence

Item	Definition / scope
Host in-memory timing	Prepared host input invocation to host-observed FPGA verdict. Full input transferred on every call.
Primary data shown	4 synthetic sizes + 8 real bitstreams; 200 measured trials per implementation/input after 20 warmups.
FPGA labels	Buffering = OPT_0; Streaming = OPT_1; Resource-optimized streaming / FPGA_HLS4ML = OPT_6 combined FIFO1024 + aggressive B2.
GPU labels	GPU_HOST = OPT_7; GPU_P2P = OPT_8. Both use original FP32 TensorFlow model.
Board / host	hacc-gpu-u55c-01; U55C device2 at 81:00.0; MI210 GPU0 at 03:00.0; CPU64 / NUMA1; GPU on NUMA0.
Correctness	520-call per-image admission. FPGA logits exact; GPU scores within CPU FP32 tolerances 1e-4 absolute/relative, exact verdicts.
Stability	Closing Streaming baseline had at most 0.051% host in-memory median drift in the full campaign.
Scope limits	Fixed run order; one session and device pairing; different model precisions. No power, accuracy-equivalence, SSD, or concurrent-load claims.

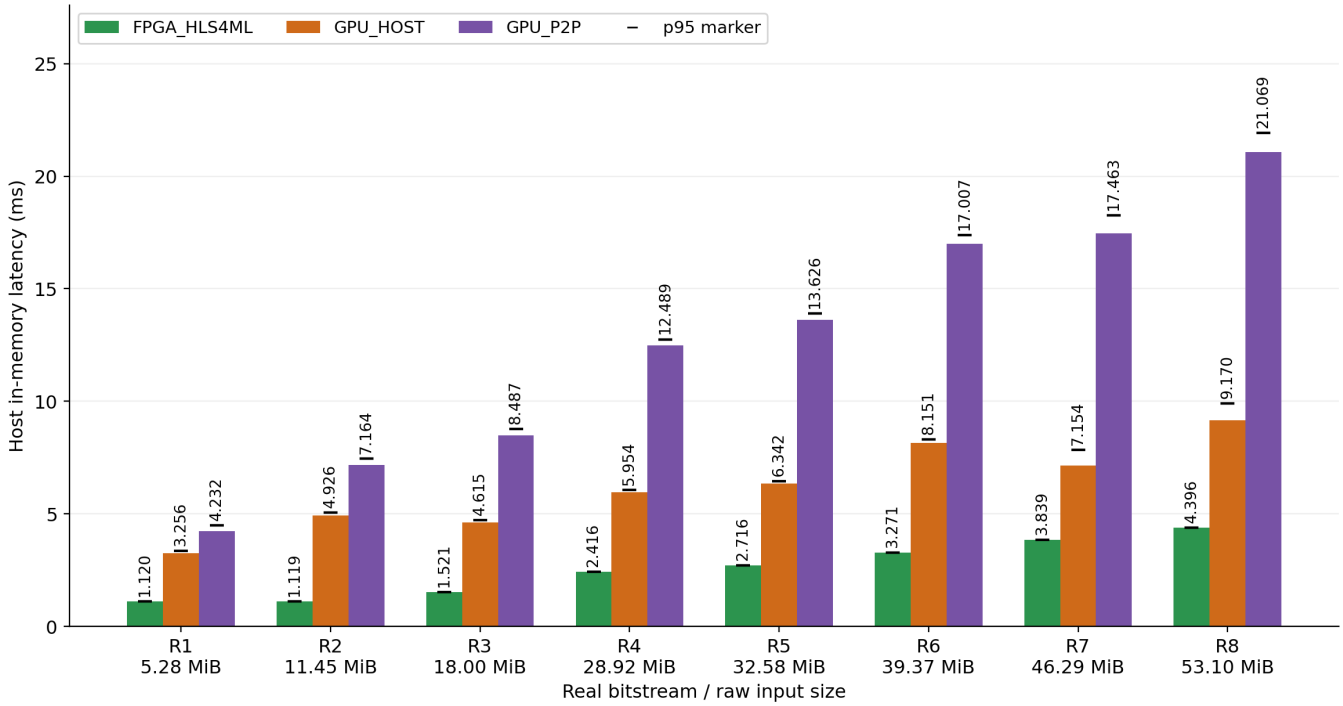
The report reads the accepted campaign manifest and evidence paths. It excludes warmups and diagnostic attempts. Main statistics use host in-memory measurements only; separate GPU profiles appear only in Figure 6. The original report remains unchanged and contains the broader measurement record.

Source: hls4ml/optimize/runs/final_benchmarks_gpu_u55c_20261019. Every source-manifest hash is checked; selected statistics are recomputed from trial data. All 60 selected implementation/input cells have 200 measured trials. No benchmark, firmware, model, or runtime was changed to produce this report.

Reproduction: run **ml_baseline/.venv/bin/python hls4ml/optimize/reports/optimizations/focused_hls4ml_checker_20261021/build_report.py** from the repository root. Figures are exported as SVG, PNG, and PDF; data/ contains statistics, ratios, phases, profiles, and representative timelines. The setup paths are also exported individually for presentations.

Appendix A1 · Real-bitstream latency

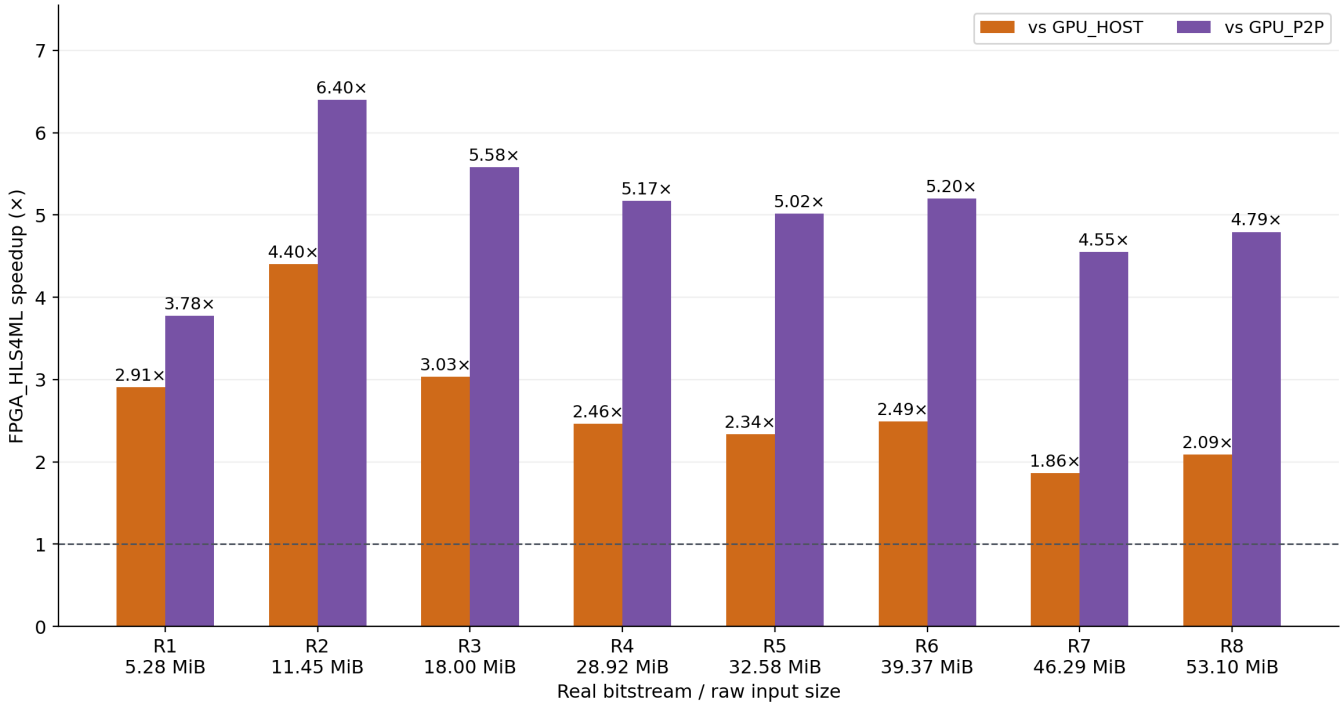
Eight real files, ordered by size. Input identity and size are listed on the final appendix page. This is the same host in-memory timing boundary used in the main figures.



Each group compares FPGA_HLS4ML, GPU_HOST, and GPU_P2P for exactly the same file. Labels show medians; black ticks show p95. Different files also have different sizes, so this is not a controlled experiment isolating the effect of bitstream contents.

Keep these measured values distinct from the synthetic-size guide curves. Host/GPU runtime effects can make latency non-monotonic between files; the evidence does not prove that real bitstream contents inherently require more computation.

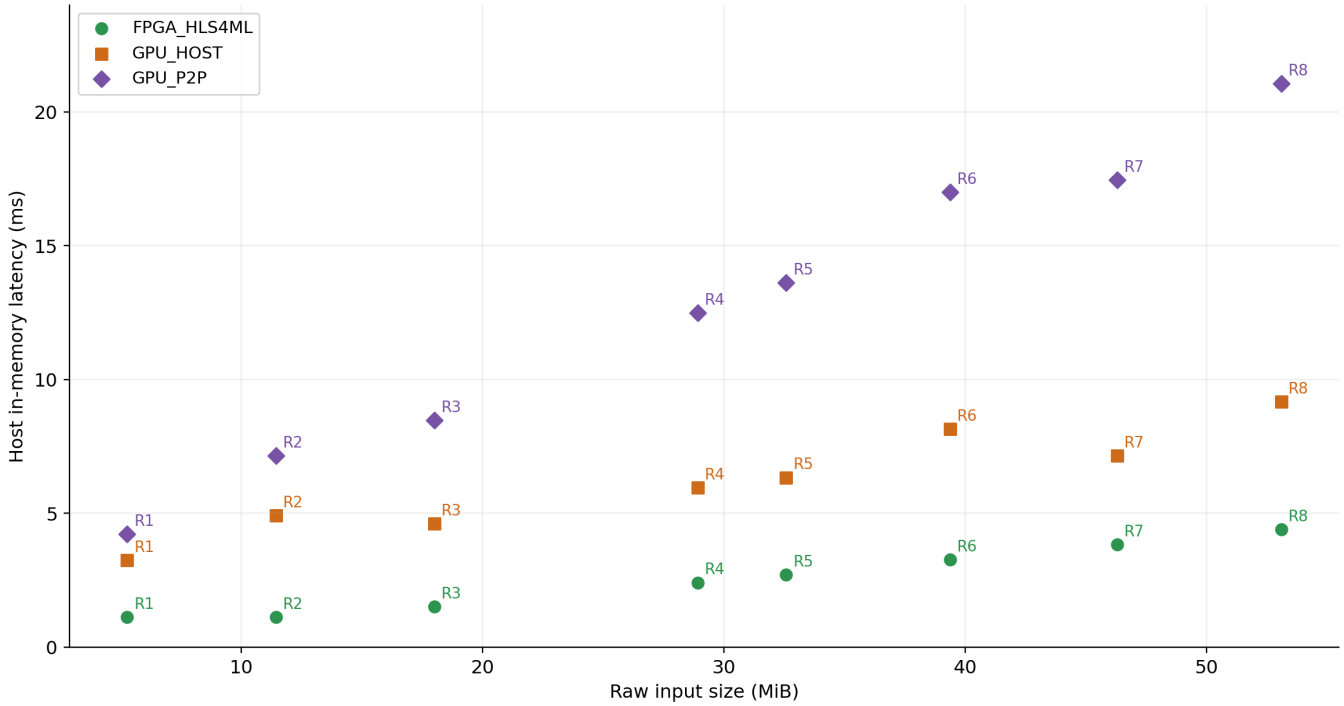
Appendix A2 · Real-bitstream FPGA speedup



For each file, speedup is the GPU path's median divided by FPGA_HLS4ML's median. Ratios above 1 favor the hls4ml-checker. All ratios use matching input files, timing scope, and the accepted host in-memory evidence.

There is no speedup scatter plot in this report. The paired bars retain the direct per-file comparison and keep the two GPU baselines explicit.

Appendix A3 · Real-bitstream latency by size



Linear size axis; unconnected measured medians. R1–R8 identify the files below. No synthetic interpolation or fitted trend is overlaid.

ID	Bytes / MiB	Source identifier
R1	5,540,996 / 5.28	HR/build/bitstreams/config_0/vfpga_c0_0.bin
R2	12,004,440 / 11.45	H4/build/bitstreams/config_0/vfpga_c0_0.bin
R3	18,876,484 / 18.00	H16/build/bitstreams/config_0/vfpga_c0_0.bin
R4	30,323,588 / 28.92	full_dataset_it1/artifacts/bitstreams/config_027/vfpga_c027_0.bin
R5	34,162,708 / 32.58	full_dataset_it1/artifacts/bitstreams/config_109/vfpga_c109_0.bin
R6	41,278,872 / 39.37	full_dataset_it2/artifacts/bitstreams/config_105/vfpga_c105_0.bin
R7	48,540,364 / 46.29	full_dataset_it4_big_ro/artifacts/bitstreams/config_050/vfpga_c050_0.bin
R8	55,676,772 / 53.10	full_dataset_it4_big_ro/artifacts/bitstreams/config_075/vfpga_c075_0.bin